

项目技术报告

背景与算法设计：

随着高校图书馆数字化建设的不断推进，借阅数据成为反映用户阅读兴趣和行为习惯的重要资源。图书馆积累了大量的借阅记录，如何利用这些数据挖掘用户的潜在需求并进行精准的图书推荐，已成为提升图书馆服务质量和用户体验的关键问题。具体任务为用户兴趣建模：基于用户的借阅历史数据，分析其阅读偏好和行为模式，构建用户兴趣特征。根据用户兴趣特征和图书信息，预测用户未来可能感兴趣的图书并生成推荐列表。

根据对问题和任务的描述，在大致研究数据集之后发现一些普遍规律，就是同一天同一个人非常喜欢借阅连号书籍，除此之外，其他时段借阅数据大多都是有规律可循，比方说一个艺术学院的同学大致上喜欢借阅四类书籍包括专业强相关的书籍，个人喜欢看的课外书，日语考级相关辅导书，考研相关书籍，而且大多时候借阅倾向倾向于某一大类。基于以上分析，我们进行了以下算法设计：

inter_id	user_id	book_id	借阅时间
69606	1	28425	2022-06-05 17:35:41
34768	1	30344	2022-06-05 17:36:05
33756	1	21185	2022-06-05 17:36:13
33758	1	21187	2022-06-05 17:36:19
33757	1	21186	2022-06-05 17:36:30
33759	1	21188	2022-06-05 17:36:41

整体上分为两种可选方案词向量模型+LSTM 模型，词向量模型+BERT 模型。

为什么要设计 LSTM 模型呢？我们可以把借阅数据看成两大类，即长时记忆数据和短时记忆数据，长时记忆数据的内涵为某一用户所有的历史借阅数据，短时记忆数据的内涵为某一用户最近几次的借阅数据。选用词向量模型是为了将书名向量化，便于计算相似度。为什么要设计 BERT 模型呢？算是对 LSTM 模型的改良

使用自注意力机制可以更好的学习特征。

我们对需要学习传入模型的特征做了一些处理，主要包括书名词向量，基于续借次数设计的交互权重，基于用户所在部门偏好设计了 dept_book_preference 和 dept_cat1_preference,book_id 本身。在传入样本数据时是按照以用户分组，借阅时间排序的数据组为单个样本

模型介绍与架构图：

词向量模型：

词向量模型是将文本中的词语转化为低维稠密数值向量的技术，核心作用是捕捉词语的语义和语法关系。如下图所示

核心特点：

- 解决传统 one-hot 编码的稀疏性问题，用固定长度向量（通常几十到几百维）高效表示词语。
- 向量空间中距离相近对应语义相似，比如 “医生” 与 “护士”、“苹果” 与 “香蕉” 的向量会靠近。

- 统一架构：无论何种 NLP 任务，均采用相同的基础架构，仅通过输出层适配，通用性高。

基本架构：

基础组件：堆叠多层 Transformer 编码器（标准 BERT-base 为 12 层，BERT-large 为 24 层）。

核心模块：

多头自注意力机制：并行计算词语与文本中所有其他词语的依赖关系，捕捉长距离语义关联。

前馈神经网络（Feed-Forward）：对每个词语的注意力输出进行非线性变换，增强模型表达能力。

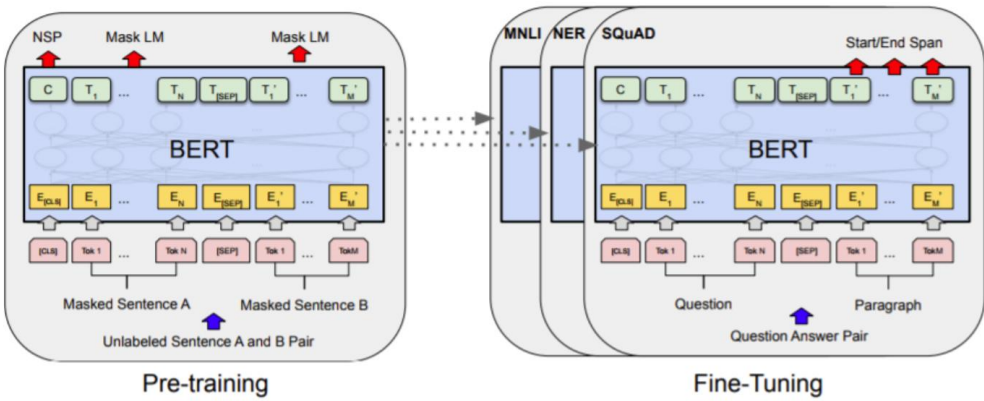
层归一化（Layer Normalization）+ 残差连接：稳定训练过程，缓解梯度消失问题。

输入表示：将词语嵌入、段落嵌入（区分不同句子）、位置嵌入（捕捉语序）三者相加，形成统一输入向量。

关键预训练任务：

Masked LM（掩码语言模型）：随机掩盖文本中 15% 的词语，让模型预测被掩盖的原词，强制模型学习上下文语义关联。

Next Sentence Prediction（下一句预测）：判断两个句子是否为连续的上下文，让模型学习句子间的逻辑关系。



上图为 BERT 原始论文所展示的基本架构

优化器为 adam

数据预处理方法：

主要由三个 python 文件实现 data_cleaning.py、data_splitting.py、feature_engineering.py。

在 data_cleaning.py 中对原始数据进行了预处理，对于用户表按分组逻辑填充缺失的年级/院系，确保 user_id 格式为字符串；对于图书表填充缺失的二级分类（利用“相邻行二级分类相同”的排序特性，前向填充）；对于借阅数据表适配借阅时间全缺失、续借逻辑自治。

136 邓小平传	《英》 理查德·伊文思著	国际文化出版公司	马克思主义、列宁主义、毛泽东思想 邓小平
137 邓小平：改革是中国的第二次革命	《邓小平：改革是中国的第二次革命》编写组著	台海出版社	马克思主义、列宁主义、毛泽东思想 <null>
138 历史转折中的邓小平	龙平平 ... 【等】 著	四川人民出版社	马克思主义、列宁主义、毛泽东思想 <null>
139 中国之谜：小平出山	薛庆超著	湖南人民出版社	马克思主义、列宁主义、毛泽东思想 <null>
140 邓小平实录	于俊道主编	中国工人出版社	马克思主义、列宁主义、毛泽东思想 <null>
141 历史选择了邓小平，第3版	高松著	武汉出版社	马克思主义、列宁主义、毛泽东思想 <null>
142 邓小平的三落三起，第3版	赵晓光，刘杰著	辽宁人民出版社	马克思主义、列宁主义、毛泽东思想 <null>
143 邓小平视察纪实	刘会田，张爱莉著	江苏人民出版社	马克思主义、列宁主义、毛泽东思想 <null>
144 邓小平实录，第2版	于俊道主编	中国工人出版社	马克思主义、列宁主义、毛泽东思想 <null>

在 data_splitting.py 中主要完成了两个任务。一个数据洗牌操作，用于在数据分割之前完成，目的是为了以防模型过拟合，第二个是数据合并分割，训练集验证集测试集的分割比例为 8：1：1。

在 feature_engineering.py 文件中主要完成一些特征工程提取时间特征：借阅月份、是否学期内（适配场景化兴趣）、提取时间特征：借阅月份、是否学期内（适配场景化兴趣）、编码分类特征：用户院系、图书分类（适配模型输入）、计算每个院系对图书的偏好度（借阅频次占比）、计算每个院系对图书分类（一级/二级）的偏好度。

借阅月份 ▾ ⚙	是否学期内 ▾ ⚙	交互权重 ▾ ⚙	DEPT ▾ ⚙	dept_book_preference ▾ ⚙	dept_cat1_preference ▾ ⚙
8	0	1.4	电子科学与工程	0.9999999900000002	0.05307855626308182

超参数选择：

有以下可选项：

模型预测与训练按钮-1=训练 LSTM 模型，2=加载 LSTM 模型并预测，3=训练 BERT 模型，4=加载 BERT 模型并预测

按钮一：

是否使用 K 折交叉验证？(1=是,0=否)：0

请输入训练轮数(默认 30)：400

请输入序列长度(默认 5)：4

请输入嵌入维度(默认 128)：128

请输入隐藏层维度(默认 256)：256

请输入 LSTM 层数(默认 2)：1

请输入 dropout 比例(默认 0.3)：0.3

请输入批次大小(默认 32)：32

请输入学习率(默认 0.001)：0.001

请输入早停耐心值(默认 3)：150

按钮二：

请输入 LSTM 模型路径（默认：model/saved_models/lstm_best_epoch167_f10.0049.pth）：

按钮三：

是否使用 K 折交叉验证？(1=是,0=否)：0

请输入序列长度(默认 5)：4

请输入单本书名最大分词长度(默认 10)：10

请输入融合层隐藏维度(默认 256)：256

请输入 dropout 比例(默认 0.3)：0.3

请输入批次大小(默认 16，BERT 建议 smaller)：32

请输入学习率(默认 2e-5，BERT 建议较小):2e-5

请输入训练轮数(默认 10)：300

请输入早停耐心值(默认 3)：200

是否冻结 BERT 权重(1=是,0=否，小数据建议冻结)：1

按钮四：

请输入 BERT 模型路径（默认：model/saved_models/bert_best_epoch10_f10.0085.pth）：

性能分析：

综合使用精确率（P）和召回率（R）计算 F1 分数

```
INFO - LSTM模型训练完成: cuda, 书名向量维度: 128
Epoch 2 - 损失: 10.1453 | 准确率: 0.0010 | F1分数: 0.0010 | 用时: 45.14秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch2_f10.0010.pth (F1: 0.0010)
Epoch 11 - 损失: 10.1447 | 准确率: 0.0031 | F1分数: 0.0031 | 用时: 44.39秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch11_f10.0031.pth (F1: 0.0031)
Epoch 20 - 损失: 10.1389 | 准确率: 0.0042 | F1分数: 0.0042 | 用时: 86.92秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch20_f10.0042.pth (F1: 0.0042)
Epoch 21 - 损失: 10.1378 | 准确率: 0.0043 | F1分数: 0.0043 | 用时: 88.94秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch21_f10.0043.pth (F1: 0.0043)
INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch27_f10.0084.pth (F1: 0.0084)
Epoch 29 - 损失: 10.1033 | 准确率: 0.0094 | F1分数: 0.0094 | 用时: 44.78秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch29_f10.0094.pth (F1: 0.0094)
Epoch 30 - 损失: 10.0945 | 准确率: 0.0115 | F1分数: 0.0115 | 用时: 45.03秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch30_f10.0115.pth (F1: 0.0115)
Epoch 33 - 损失: 10.0312 | 准确率: 0.0336 | F1分数: 0.0336 | 用时: 44.65秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch33_f10.0336.pth (F1: 0.0336)
Epoch 34 - 损失: 10.0493 | 准确率: 0.0357 | F1分数: 0.0357 | 用时: 63.63秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch34_f10.0357.pth (F1: 0.0357)
Epoch 37 - 损失: 10.0108 | 准确率: 0.0368 | F1分数: 0.0368 | 用时: 87.18秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch37_f10.0368.pth (F1: 0.0368)
Epoch 38 - 损失: 9.9974 | 准确率: 0.0388 | F1分数: 0.0388 | 用时: 90.52秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch38_f10.0388.pth (F1: 0.0388)
Epoch 39 - 损失: 9.9841 | 准确率: 0.0399 | F1分数: 0.0399 | 用时: 88.71秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch39_f10.0399.pth (F1: 0.0399)
Epoch 40 - 损失: 9.9704 | 准确率: 0.0230 | F1分数: 0.0230 | 用时: 71.34秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch40_f10.0230.pth (F1: 0.0230)
Epoch 46 - 损失: 9.8830 | 准确率: 0.0283 | F1分数: 0.0283 | 用时: 45.55秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch46_f10.0283.pth (F1: 0.0283)
Epoch 49 - 损失: 9.8310 | 准确率: 0.0293 | F1分数: 0.0293 | 用时: 46.69秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch49_f10.0293.pth (F1: 0.0293)
Epoch 51 - 损失: 9.7926 | 准确率: 0.0346 | F1分数: 0.0346 | 用时: 44.92秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch51_f10.0346.pth (F1: 0.0346)
Epoch 53 - 损失: 9.7557 | 准确率: 0.0356 | F1分数: 0.0356 | 用时: 45.06秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch53_f10.0356.pth (F1: 0.0356)
Epoch 54 - 损失: 9.7255 | 准确率: 0.0366 | F1分数: 0.0366 | 用时: 45.06秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch54_f10.0366.pth (F1: 0.0366)
Epoch 55 - 损失: 9.7167 | 准确率: 0.0377 | F1分数: 0.0377 | 用时: 45.23秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch55_f10.0377.pth (F1: 0.0377)
Epoch 56 - 损失: 9.6974 | 准确率: 0.0387 | F1分数: 0.0387 | 用时: 45.58秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch56_f10.0387.pth (F1: 0.0387)
Epoch 57 - 损失: 9.6768 | 准确率: 0.0429 | F1分数: 0.0429 | 用时: 45.12秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch57_f10.0429.pth (F1: 0.0429)
Epoch 58 - 损失: 9.6577 | 准确率: 0.0440 | F1分数: 0.0440 | 用时: 45.12秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch58_f10.0440.pth (F1: 0.0440)
Epoch 60 - 损失: 9.6188 | 准确率: 0.0450 | F1分数: 0.0450 | 用时: 45.20秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch60_f10.0450.pth (F1: 0.0450)
Epoch 61 - 损失: 9.5986 | 准确率: 0.0545 | F1分数: 0.0545 | 用时: 45.07秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch61_f10.0545.pth (F1: 0.0545)
Epoch 68 - 损失: 9.4691 | 准确率: 0.0586 | F1分数: 0.0586 | 用时: 45.02秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch68_f10.0586.pth (F1: 0.0586)
Epoch 77 - 损失: 9.3473 | 准确率: 0.0597 | F1分数: 0.0597 | 用时: 45.11秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch77_f10.0597.pth (F1: 0.0597)
Epoch 78 - 损失: 9.3373 | 准确率: 0.0607 | F1分数: 0.0607 | 用时: 45.11秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch78_f10.0607.pth (F1: 0.0607)
Epoch 83 - 损失: 9.3083 | 准确率: 0.0628 | F1分数: 0.0628 | 用时: 45.10秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch83_f10.0628.pth (F1: 0.0628)
Epoch 92 - 损失: 9.2711 | 准确率: 0.0702 | F1分数: 0.0702 | 用时: 49.80秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch92_f10.0702.pth (F1: 0.0702)
Epoch 198 - 损失: 9.2054 | 准确率: 0.0712 | F1分数: 0.0712 | 用时: 47.78秒 INFO - 最佳lstm模型已保存至: model/saved_models/lstm_best_epoch198_f10.0712.pth (F1: 0.0712)
INFO - LSTM训练完成 - 最佳F1: 0.0712, 总耗时: 50525.97秒
Epoch 300 - 损失: 9.1870 | 准确率: 0.0513 | F1分数: 0.0513 | 用时: 45.45秒
训练完成! 总耗时: 50525.97秒
最佳lstm F1: 0.0712
```

不足与改良之处:

这两种模型虽然在执行指定的任务方面已经达到了基本的要求,但在对输出结果进行细致的检查和分析后,我们发现部分用户的预测结果存在明显的不合理之处,这无疑会对整体的预测准确性和用户体验造成一定的影响。为了有效解决这一问题,确保模型的预测结果更加精准和可靠,我们经过深入研究和探讨,提出了以下针对性的改良方案:采用 BERT+BiLSTM+CRF 的复合模型架构设计。这一方案结合了 BERT 模型在自然语言处理中的强大能力、BiLSTM 模型在序列数据处理中的优势,以及 CRF 模型在标签序列预测中的精确性,旨在全面提升模型的性能和预测结果的合理性。

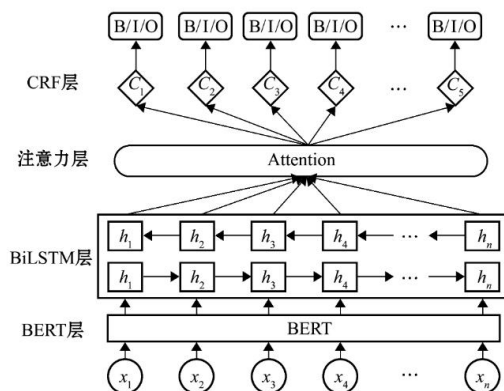


图7 BERT+BiLSTM+Attention+CRF网络模型结构

Fig.7 Model structure of BERT+BiLSTM+Attention+CRF

具体而言,BERT 模型能够深入理解书名的语义信息,生成高质量的特征向量;BiLSTM(双向长短期记忆网络)则能有效捕捉用户借阅序列中蕴含的长短期时序依赖关系,同时考虑历史借阅对未来兴趣的影响;CRF(条件随机场)层则通过建模推荐项之间的关联性(如类别连续性、主题一致性),对 BiLSTM 的输出序列施加全局约束,从而优化最终的推荐序列结构,减少推荐结果中出现逻辑跳跃或不相关图书的可能性。这种组合模型预期能够更全面地建模用户兴趣的动态演变过程,并生成逻辑更连贯、主题更聚焦的推荐列表。

我们还可以考虑给 LSTM 模型加入滑动窗口机制——这种方法的核心思想是,利用历史数据来预测未来的值。通过在时间序列中创建一个固定长度的时间窗口,提取该窗口内的特征信息来预测未来的值。随着

窗口向前滑动，我们可以得到不同的训练数据，并不断更新模型。还可以采用集成算法思想，用不同算法训练多个弱训练器等等。

此外，在特征工程方面，考虑引入更细粒度的用户画像信息，如专业方向（若数据允许）或历史借阅书籍的平均阅读时长（需日志支持），以及书籍本身的更多元属性（如作者热度、出版年份、豆瓣评分等外部知识），以丰富模型的输入特征，提升表征能力。在模型训练策略上，计划探索更精细的加权采样方法，例如对借阅频次低但重要的冷门书籍或新用户样本给予更高权重，以缓解数据稀疏性问题。同时，优化超参数搜索策略，如采用贝叶斯优化代替网格搜索，以提高调参效率并找到更优的模型配置参数。